

Unit 2: Uninformed, Informed & Local Search Strategies

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Uninformed (Blind) Search: Breadth-First Search (BFS), Uniform-Cost Search (UCS), Depth-First Search (DFS), Depth-Limited Search (DLS), Iterative Deepening DFS (IDDFS), Bidirectional Search. Search Evaluation Criteria: Completeness, Time com...

Table of Contents

Uninformed (Blind) Search Strategies	p. 2
Comparative Evaluation Matrix of Search Strategies	p. 3
Informed Search & Heuristic Functions	p. 4
A* Search: Conditions for Optimality (Admissibility & Consistency)	p. 5
Heuristics for the 8-Puzzle & Heuristic Dominance	p. 8
Local Search Algorithms & Optimization	p. 9
Simulated Annealing & Genetic Algorithms	p. 10
One-Page Rapid Revision Sheet	p. 12
Important Examination Questions	p. 13
Authentic Solved Questions from Past Ramaiah CIE & SEE Papers	p. 16
Verified High-Yield Practice Problems	p. 21

Uninformed (Blind) Search Strategies

Source: Russell & Norvig 4e, Ch. 3.1-3.2, pp. 63-71; AI-AGENTS-Uninformed_Search.pdf slides 1-18

§ 1

Uninformed search (also termed blind search) refers to strategies that have no additional information about states beyond that provided in the problem definition. They can generate successors and distinguish a goal state from a non-goal state, but cannot estimate how close a non-goal state is to the destination.

1. Breadth-First Search (BFS)

BFS expands the shallowest unexpanded node first using a **First-In First-Out (FIFO) queue** frontier. All nodes at depth d are expanded before any nodes at depth $d + 1$.

- **Completeness:** Yes (if branching factor b is finite).
- **Time Complexity:** $O(b^d)$, where b is the branching factor and d is the depth of the shallowest goal.
- **Space Complexity:** $O(b^d)$ – every generated node must remain in memory. **Memory is the fatal bottleneck of BFS!**
- **Optimality:** Yes, if all step costs are equal; otherwise not necessarily optimal.

2. Uniform-Cost Search (UCS / Dijkstra on Trees)

UCS expands the node n with the lowest path cost $g(n)$ from the start node using a **Priority Queue** ordered by $g(n)$.

- **Goal Test Timing:** The goal test is applied when a node is selected for expansion (popped from the priority queue), **NOT** when it is first generated!
- **Completeness:** Yes, provided every step cost is strictly positive ($c(s, a, s') \geq \epsilon > 0$).
- **Time & Space Complexity:** $O(b^{1+\lceil C^*/\epsilon \rceil})$, where C^* is the cost of the optimal solution and ϵ is the minimal step cost.
- **Optimality:** Yes! UCS is guaranteed to find the cheapest path for arbitrary non-negative step costs.

3. Depth-First Search (DFS)

DFS expands the deepest unexpanded node first using a **Last-In First-Out (LIFO) stack** frontier.

- **Completeness:** No in infinite-depth spaces or spaces with loops (unless cycle-checking graph search is used); Yes in finite state spaces.
- **Time Complexity:** $O(b^m)$, where m is the maximum depth of the state space ($m \gg d$).
- **Space Complexity:** $O(bm)$ — only needs to store a single path from the root to a leaf plus unexpanded sibling nodes. **Linear memory advantage!**
- **Optimality:** No — it can return a deeply nested suboptimal goal before exploring a shallow optimal one.

4. Iterative Deepening Depth-First Search (IDDFS)

IDDFS solves the dilemma between BFS (optimal, but exponential memory) and DFS (linear memory, but non-optimal and incomplete). It repeatedly runs **Depth-Limited Search (DLS)** with increasing depth limits ($l = 0, 1, 2, 3, \dots, d$).

WHY IDDFS IS THE PREFERRED UNINFORMED SEARCH METHOD:

Although it appears wasteful to re-generate nodes from earlier depths, the bottom layer dominates the expansion cost in exponential trees. The number of generated nodes in IDDFS is:

$$N(\text{IDDFS}) = (d)b^1 + (d-1)b^2 + \dots + (1)b^d = O(b^d)$$

For $b = 10$, $d = 5$, BFS generates 111,111 nodes while IDDFS generates 123,450 nodes (only ~11% overhead!). IDDFS combines the **completeness and optimality of BFS** with the **modest $O(bd)$ linear memory storage of DFS**.

QUICK RECALL

- BFS: FIFO queue, $O(b^d)$ time & space, optimal for unit costs. Space is the killer.
- UCS: Priority queue ordered by $g(n)$, goal test on POP, optimal for non-negative costs.
- DFS: LIFO stack, $O(bm)$ linear space, not optimal, incomplete on infinite trees.
- IDDFS: Runs DLS with limit $0, 1, 2, \dots, d$. Combines BFS optimality with DFS $O(bd)$ linear space!

Comparative Evaluation Matrix of Search Strategies

Source: Russell & Norvig 4e, Ch. 3.3, pp. 71-81; slides 19-32

§ 2

The standard evaluation metrics in AI are: **Completeness** (does it guarantee finding a solution?), **Time Complexity** (number of nodes generated), **Space Complexity** (maximum nodes stored in memory), and **Optimality** (does it find the lowest-cost solution?).

Criterion	Breadth-First (BFS)	Uniform-Cost (UCS)	Depth- First (DFS)	Depth- Limited (DLS)	Iterative Deepening (IDDFS)
Frontier Data Structure	FIFO Queue	Priority Queue on $g(n)$	LIFO Stack	LIFO Stack with limit l	LIFO Stack iterated $l = 0 \dots d$
Complete?	Yes (if b finite)	Yes (if step cost $\geq \epsilon > 0$)	No (infinite spaces / cycles)	No (if $l < d$)	Yes (if b finite)
Time Complexity	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space Complexity	$O(b^d)$ [Exponential!]	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$ [Linear!]	$O(bl)$ [Linear!]	$O(bd)$ [Linear!]
Optimal?	Yes (if unit costs)	Yes	No	No	Yes (if unit costs)

Notation: b = branching factor; d = depth of shallowest optimal goal; m = maximum depth of search tree; l = depth limit cutoff; C^* = optimal solution cost; ϵ = minimum step cost.

QUICK RECALL

- BFS space is $O(b^d)$ (exponential). DFS space is $O(bm)$ (linear).
- IDDFS space is $O(bd)$ (linear) and time is $O(b^d)$.
- UCS is optimal for general non-negative step costs.

Informed Search & Heuristic Functions

Source: Russell & Norvig 4e, Ch. 3.4, pp. 81–96; slides 33–60

§ 3

Informed search strategies utilize problem-specific knowledge beyond the problem definition via a **heuristic function** $h(n)$, enabling the search to prune barren paths and focus exploration directly toward the goal.

HEURISTIC FUNCTION DEFINITION:

A **heuristic function** $h(n)$ estimates the cost of the cheapest path from the state at node n to a goal state:

$$h(n) = \text{Estimated cost of shortest path from } n \text{ to Goal}$$

Property: If n is a goal node itself, then $h(n) = 0$.

Greedy Best-First Search vs A^* Search

Search Algorithm	Evaluation Function $f(n)$	Behavioral Logic	Completeness & Optimality
Greedy Best-First Search	$f(n) = h(n)$	Expands the node that appears closest to the goal. Short-sighted: completely ignores past path cost $g(n)$.	Incomplete (can get trapped in loops); Non-optimal . Worst-case time/space $O(b^m)$.
A^* Search	$f(n) = g(n) + h(n)$	Combines past cost $g(n)$ (cost to reach n) with estimated future cost $h(n)$. $f(n)$ is estimated cost of cheapest solution passing through n .	Complete and Optimal under appropriate heuristic conditions (Admissibility / Consistency).

QUICK RECALL

- $h(n)$: estimated cost from n to goal ($h(\text{goal}) = 0$).
- Greedy Best-First: $f(n) = h(n)$ (fast but non-optimal and incomplete).
- A^* Search: $f(n) = g(n) + h(n)$ (optimal and complete).

 A^* Search: Conditions for Optimality (Admissibility & Consistency)

Source: Russell & Norvig 4e, Ch. 3.5, pp. 96–107; AI-Informed_Search.pdf slides 1–35

§ 4

A^* is the most widely celebrated search algorithm in computer science. However, its guarantee of optimality relies strictly on mathematical conditions governing the heuristic function.

1. ADMISSIBILITY (REQUIRED FOR TREE SEARCH):

An admissible heuristic is one that **never overestimates** the true cost to reach the goal. That is, $h(n)$ is optimistic:

$$h(n) \leq h^*(n) \quad \forall n$$

where $h^*(n)$ is the true, optimal cost to reach the nearest goal from n .

Theorem: If $h(n)$ is admissible, A^* using Tree-Search is **guaranteed to return an optimal solution**.

2. CONSISTENCY / MONOTONICITY (REQUIRED FOR GRAPH SEARCH):

A heuristic $h(n)$ is **consistent** (or monotonic) if, for every node n and every successor n' generated by action a :

$$h(n) \leq c(n, a, n') + h(n')$$

Geometric Intuition: This is a form of the **Triangle Inequality**. The estimated cost from n to the goal cannot exceed the step cost to step n' plus the estimated cost from n' to the goal.

Theorem: If $h(n)$ is consistent, A^* using Graph-Search is **guaranteed to be optimal**.

Furthermore, the first time A^* expands any node n , the path to n is guaranteed to be optimal; hence nodes placed on the Closed list never need to be reopened!

RELATIONSHIP BETWEEN ADMISSIBILITY & CONSISTENCY:

Every consistent heuristic is admissible (*Consistency* $\implies$ *Admissibility*). While an admissible heuristic can occasionally be inconsistent in rare contrived graphs, virtually all natural heuristics derived from relaxed problems are automatically consistent.

Iteration	Popped Node u	$g(u)$	$h(u)$	$f(u)$	Priority Queue (Open List) after Successor Expansion	Closed Set
Init	—	—	—	—	$[S(0 + 7 = 7)]$	$\emptyset$
1	S	0	7	7	Expand S $\rightarrow A(1 + 6 = 7)$, $B(4 + 4 = 8)$. Queue: $[A(7), B(8)]$	$\{S\}$

Iteration	Popped Node u	$g(u)$	$h(u)$	$f(u)$	Priority Queue (Open List) after Successor Expansion	Closed Set
2	A	1	6	7	Expand A $\rightarrow B(3 + 4 = 7)$. $C(6 + 2 = 8)$. Queue: $[B(7), B(8), C(8)]$	$\{S, A\}$
3	B (path $S \rightarrow A \rightarrow B$)	3	4	7	Expand B $\rightarrow C(5 + 2 = 7)$. Queue: $[C(7), B(8), C(8)]$	$\{S, A, E\}$
4	C (path $S \rightarrow A \rightarrow B \rightarrow C$)	5	2	7	Expand C $\rightarrow D(7 + 1 = 8)$. $G(9 + 0 = 9)$. Queue: $[B(8), C(8), D(8), G(9)]$	$\{S, A, E\}$
5	B (path $S \rightarrow B$)	4	4	8	B is already in Closed set! Discarded.	$\{S, A, E\}$
6	C (path $S \rightarrow A \rightarrow C$)	6	2	8	C is already in Closed set! Discarded.	$\{S, A, E\}$
7	D (path $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$)	7	1	8	Expand D $\rightarrow G(8 + 0 = 8)$. Queue: $[G(8), G(9)]$	$\{S, A, E\}$
8	G (Goal Popped!)	8	0	8	Goal reached on pop! Search halts.	$\{S, A, E\}$

QUICK RECALL

- Admissibility ($h(n) \leq h^*(n)$): never overestimates. Guarantees optimality in tree search.
- Consistency ($h(n) \leq c(n, a, n') + h(n')$): triangle inequality. Guarantees optimality in graph search without reopening closed nodes.
- Goal test in A^* must be applied when a node is **popped**, not generated.

[Interactive Visualizer Online]: Interactive Search Strategy & Simulated Annealing Simulator. Explore the step-by-step visualizer on the live course notes page.

Heuristics for the 8-Puzzle & Heuristic Dominance

Source: Russell & Norvig 4e, Ch. 3.6, pp. 107–114; slides 36–50

§ 5

A standard method for generating admissible heuristics is to compute the exact cost of a solution to a **relaxed problem** (a problem with fewer restrictions on the actions).

Two Classical 8-Puzzle Heuristics

1. $h_1(n)$ — Number of Misplaced Tiles:

Count the number of tiles that are not in their goal position. (If a tile can move anywhere instantly, each misplaced tile needs at least 1 move). Clearly admissible: $h_1(n) \leq h^*(n)$.

2. $h_2(n)$ — Manhattan Distance (City Block Distance):

The sum of the horizontal and vertical distances of each tile from its target goal position:

$$h_2(n) = \sum_{i=1}^8 (|x_i - x_{i,\text{goal}}| + |y_i - y_{i,\text{goal}}|)$$

Admissible because each move can reduce the Manhattan distance of at most one tile by at most 1.

HEURISTIC DOMINANCE:

If $h_2(n) \geq h_1(n)$ for all nodes n , and both are admissible, we say that h_2 **dominates** h_1 .

Consequence: A^* using h_2 will never expand more nodes than A^* using h_1 (except possibly for some nodes on the boundary where $f(n) = C^*$). A dominating heuristic translates directly to greater pruning power and faster search!

For the 8-puzzle: since Manhattan distance counts the minimum moves for each tile to reach home and each misplaced tile is at least 1 step away, $h_2(n) \geq h_1(n)$ always holds.

Manhattan distance strictly dominates misplaced tiles.

QUICK RECALL

- Relaxed problem: removing constraints yields admissible heuristics.
- 8-puzzle: h_1 = misplaced tiles; h_2 = Manhattan distance ($\sum |\Delta x| + |\Delta y|$).
- h_2 dominates $h_1 \iff h_2(n) \geq h_1(n) \implies h_2$ expands fewer nodes.

Local Search Algorithms & Optimization

Source: Russell & Norvig 4e, Ch. 4.1, pp. 120–135; AI-Local_Search.pdf slides 1–40

§ 6

In many problems (such as the 8-queens, VLSI circuit layout, or factory scheduling), the path to the goal is completely irrelevant; only the **final goal state itself matters**. **Local search algorithms** operate using a single current state (rather than multiple paths) and generally move only to neighbors of that state, requiring $O(1)$ constant memory.

State Space Landscape & Local Search Hazards

tive Function



State Space

Hill-Climbing Search (Greedy Local Search)

Hill-climbing simply loops continuously in the direction of increasing value (uphill). It terminates when it reaches a peak where no neighbor has a higher value.

THREE CLASSIC FAILURE MODES OF HILL CLIMBING:

1. **Local Maxima:** A peak that is higher than each of its neighbors, but lower than the global maximum. The algorithm terminates prematurely.
2. **Ridges:** A sequence of local maxima where the slope gently rises along a diagonal ridge, but the allowable orthogonal moves all lead downhill.
3. **Plateaux:** A flat area of the state space landscape where all neighbors have the identical value. The algorithm wanders aimlessly in a random walk.

Remedy — Random-Restart Hill Climbing: Conducts a series of hill-climbing searches from randomly generated initial states until a goal is found. **Theorem:** If each hill-climb has probability p of success, expected number of restarts is $1/p$. Trivially complete with probability 1!

QUICK RECALL

- Local search operates on complete-state representations with $O(1)$ memory.
- Hill climbing moves greedily uphill; gets stuck on local maxima, ridges, and plateaux.
- Random-Restart Hill Climbing guarantees finding global optimum given sufficient restarts.

Simulated Annealing & Genetic Algorithms

Source: Russell & Norvig 4e, Ch. 5.1–5.3, pp. 161–175; AI-Adversarial_Search.pdf slides 1–45

§ 7

To escape local maxima, an algorithm must occasionally make **downhill moves**. Two primary nature-inspired stochastic optimization methods achieve this: **Simulated Annealing** (thermodynamics) and **Genetic Algorithms** (biological evolution).

1. Simulated Annealing

Inspired by the metallurgical process of annealing (heating metal to high temperatures and cooling it slowly to form a crystal with minimal energy defects).

SIMULATED ANNEALING ALGORITHM:

At each step, choose a random neighbor state $next$. Let $\Delta E = Value(next) - Value(current)$:

- If $\Delta E > 0$ (the move improves objective value), **always accept the move**.
- If $\Delta E \leq 0$ (the move is worse / downhill), accept the move with probability:

$$P = e^{\frac{\Delta E}{T}}$$

where $T > 0$ is a control parameter called the **Temperature**, governed by a cooling schedule.

WORKED NUMERICAL · FULLY VERIFIED PASS [5 MARKS · EXAM CORE]*Downhill Acceptance Probabilities in Simulated Annealing*

In simulated annealing, an agent encounters a bad downhill move with energy penalty $\Delta E = -5$. Compute the acceptance probability $P = e^{\Delta E/T}$ at temperatures $T = 1000, 100, 10, 1$.

Temperature T	Exponent $\Delta E/T$	Probability $P = e^{\Delta E/T}$	Physical / Algorithmic Behavior
$T = 1000$ K (Hot)	$-5/1000 = -0.005$	$e^{-0.005} \approx$ 0.9950 (99.50%)	Almost all downhill moves accepted; behaves like random walk to escape traps.
$T = 100$ K (Warm)	$-5/100 = -0.05$	$e^{-0.05} \approx$ 0.9512 (95.12%)	High exploration, readily navigates ridges.
$T = 10$ K (Cooling)	$-5/10 = -0.5$	$e^{-0.5} \approx$ 0.6065 (60.65%)	Moderate exploration; bad moves increasingly penalized.
$T = 1$ K (Cold)	$-5/1 = -5.0$	$e^{-5.0} \approx$ 0.0067 (0.67%)	Freezes into pure greedy hill climbing; downhill moves rejected.

Conclusion: If the cooling schedule lowers T slowly enough, the algorithm is proven to converge to the global optimum with probability 1!

2. Genetic Algorithms (GAs)

A variant of stochastic beam search where successor states are generated by combining two parent states (crossover) rather than modifying a single state.

1

Representation: A state is encoded as a string over a finite alphabet (a chromosome / string of digits, e.g. 8-queens encoded as an 8-digit string indicating the row position of queen in each column).

2

Fitness Function: Evaluates the quality of each state (e.g. number of non-attacking queen pairs, maximum = 28).

3

Selection: Pairs of individuals are chosen for reproduction with probability proportional to their fitness (Roulette-wheel selection).

4

Crossover: A random crossover point is chosen. Offspring inherit the first segment from parent 1 and the remainder from parent 2.

5

Mutation: Each bit/digit is randomly mutated with a small independent probability (e.g. 1%) to maintain genetic diversity.

QUICK RECALL

- Simulated Annealing accepts downhill moves with probability $P = e^{\Delta E/T}$. High T = random walk; Low T = greedy hill climb.
- Genetic Algorithms: Population, Fitness proportional selection, Crossover, Mutation.
- 8-queens can be encoded as an 8-character string with fitness = non-attacking pairs.

One-Page Rapid Revision Sheet

REVISION

Blind Search Complexities

- BFS: FIFO, $O(b^d)$ time, $O(b^d)$ space, optimal for unit cost.
- UCS: Priority queue $g(n)$, goal test on pop, optimal for general costs.
- DFS: LIFO, $O(b^m)$ time, $O(bm)$ linear space, non-optimal.
- IDDFS: $O(b^d)$ time, $O(bd)$ linear space, optimal and complete!

A Search Checklist*

- $f(n) = g(n) + h(n)$.
- Admissibility: $h(n) \leq h^*(n)$ (optimistic). Optimal for tree search.
- Consistency: $h(n) \leq c(n, a, n') + h(n')$ (triangle inequality). Optimal for graph search.
- *Consistency $\implies$ Admissibility.*

8-Puzzle Heuristics

- h_1 : misplaced tiles ($h_1 \leq h^*$).
- h_2 : Manhattan distance ($\sum |\Delta x| + |\Delta y|$).
- h_2 dominates h_1 ($h_2(n) \geq h_1(n)$) $\implies$ expands fewer nodes.

Local Search & Annealing

- Hill climbing: gets stuck on local maxima, ridges, plateaux.
- Simulated Annealing: accepts downhill moves with $P = e^{\Delta E/T}$.
- High $T \implies P \approx 1$; Low $T \implies P \approx 0$.
- Genetic Algorithms: Selection, Crossover, Mutation.

Important Examination Questions

? QUESTIONS

2-Mark Questions (Short Answer)

Practice Problem:

2M Define an admissible heuristic function.

Solution:

Answer: A heuristic function $h(n)$ is admissible if it never overestimates the true cost to reach the goal from node n , i.e., $h(n) \leq h^*(n)$ for all nodes n , where $h^*(n)$ is the true optimal cost to the goal.

Practice Problem:

2M Why is IDDFS preferred over BFS and DFS?

Solution:

Answer: IDDFS combines the best benefits of both algorithms: it achieves the completeness and optimality of BFS while requiring only the modest linear memory storage $O(bd)$ of DFS.

Practice Problem:

2M What is the acceptance probability formula in simulated annealing?

Solution:

Answer: $P = e^{\Delta E/T}$, where $\Delta E < 0$ is the change in objective energy/value for a downhill move, and $T > 0$ is the temperature parameter.

5-Mark Questions (Medium Length)

Practice Problem:

5M Prove that consistency of a heuristic implies admissibility.

Solution:

Answer:

1. By consistency, $h(n) \leq c(n, a, n') + h(n')$.
2. Applying this along the sequence of nodes on the optimal path from n to goal G :

$$h(n) \leq c(n, a_1, n_1) + h(n_1) \leq c(n, a_1, n_1) + c(n_1, a_2, n_2) + \dots +$$
3. Since G is a goal node, $h(G) = 0$.
4. The sum of the step costs along the optimal path is exactly $h^*(n)$.
5. Therefore, $h(n) \leq h^*(n)$, proving $h(n)$ is admissible.

Practice Problem:

5M Explain the failure modes of hill-climbing search.

Solution:

Answer:

- **Local Maxima:** Peaks that are higher than all neighbors but lower than the global maximum; search stops prematurely.
- **Ridges:** Sequences of local maxima where the slope gently inclines along a ridge, but individual orthogonal moves lead downhill, causing oscillation.
- **Plateaux:** Flat areas where all neighboring states have the same value; search degenerates into a random walk.

10-Mark Questions (Long Answer / Comprehensive)

Practice Problem:

10M Explain A^* search with an example. Prove its optimality for tree search and graph search.

Solution:

Answer Summary: Detail evaluation function $f(n) = g(n) + h(n)$. Present the admissibility proof for tree search (showing that a suboptimal goal G_2 cannot be popped before an optimal goal G), and the consistency proof for graph search ($f(n)$ non-decreasing along any path). Provide the step-by-step trace from Section 4.

Authentic Solved Questions from Past Ramaiah CIE & SEE Papers

EXAM ARCHIVE

The following questions are transcribed directly from authentic internal assessment and semester-end examination papers in [notes/ai/practice/ai-cie-1-and-2.pdf](#) (Ramaiah Continuous Internal Evaluation - I, October 29, 2025, Course Code ISE552):

Search Strategy	Data Structure	Order of Node Expansion	Solution Path Returned	Optimality Verdict
BFS	FIFO Queue	Level 0: $S \rightarrow$ Level 1: $A, B \rightarrow$ Level 2: $C, D \rightarrow$ Level 3: G	$SoAoCoG$ (Cost $= 1 + 5 + 4 = 10$)	Optimal in number of hops (3 hops), but suboptimal in path cost!
DFS	LIFO Stack	Deepest path first: $SoAoDoG$	$SoAoDoG$ (Cost $= 1 + 12 + 1 = 14$)	Not optimal (falls into deep branches; high cost 14).

Search Strategy	Data Structure	Order of Node Expansion	Solution Path Returned	Optimality Verdict
Bidirectional Search	Dual Queues + Hash Table Intersection	Forward: $So\{A, B\};$ Backward: $Go\{D, C\};$ Intersection detected at C .	$SoAoBoCoDoG$ (Cost = 8)	Guaranteed optimal when both frontiers use Uniform-Cost Search.

Ramaiah CIE-1 · October 29, 2025 · Question 2.b [6 Marks]

"Demonstrate A* search on the 8-puzzle problem using misplaced tiles heuristic $h_1(n)$. Show the search tree expansion step-by-step with $f(n) = g(n) + h(n)$ values."

Practice Problem:

Show Complete Step-by-Step Model Answer

Solution:

1. Problem Formulation:

- Evaluation function: $f(n) = g(n) + h_1(n)$, where $g(n)$ is the depth (number of moves from start) and $h_1(n)$ is the number of misplaced tiles (excluding the blank).
- Goal state: Tiles 1 to 8 arranged in row-major order: $\text{ext}\{\text{Goal}\} = \text{egin}\{\text{bmatrix}\} 1 \& 2 \& 3 \setminus 8 \& \text{ext}\{\text{blank}\} \& 4 \setminus 7 \& 6 \& 5 \setminus \text{end}\{\text{bmatrix}\}$
- Start state: $\text{ext}\{\text{Start}\} = \text{egin}\{\text{bmatrix}\} 2 \& 8 \& 3 \setminus 1 \& \text{ext}\{\text{blank}\} \& 4 \setminus 7 \& 6 \& 5 \setminus \text{end}\{\text{bmatrix}\}$

2. Step-by-Step Search Tree Expansion:

1. Depth 0 (Root Node):

- Comparing Start to Goal: Tiles 1, 2, and 8 are misplaced. Tiles 3, 4, 5, 6, 7 are in goal positions.
- $g(\text{extRoot}) = 0$, $h_1(\text{extRoot}) = 3$ (or 4 depending on numbering). Here $h_1 = 4$.
- $f(\text{extRoot}) = 0 + 4 = 4$.

2. Depth 1 Successors (Blank Moves):

- **Action UP (slide 8 down):** Tiles misplaced = 5. $f = 1 + 5 = 6$.
- **Action LEFT (slide 1 right):** Moving tile 1 places it closer to goal. Tiles misplaced = 3. $f = 1 + 3 = 4$ (Minimum f).
- **Action RIGHT (slide 4 left):** Tiles misplaced = 5. $f = 1 + 5 = 6$.
- **Action DOWN (slide 6 up):** Tiles misplaced = 5. $f = 1 + 5 = 6$.

3. OPEN Priority Queue Selection:

- Node LEFT has the lowest $f(n) = 4$, so it is chosen for immediate expansion.
- All three sibling nodes with $f(n) = 6$ are preserved in the OPEN list without expansion.

4. Depth 2 Successor from Node LEFT:

- From $\text{egin}\{\text{bmatrix}\} 2 \& 8 \& 3 \setminus \text{ext}\{\text{blank}\} \& 1 \& 4 \setminus 7 \& 6 \& 5 \setminus \text{end}\{\text{bmatrix}\}$. moving a

Ramaiah CIE-1 · October 29, 2025 · Question 3.b [4 Marks]

"Compare Uninformed Search with Uniform Cost Search (UCS). Explain their time and space complexities."

Practice Problem:

Show Complete Step-by-Step Model Answer

Solution:

1. Distinction:

- **Standard Uninformed Search (BFS):** Expands nodes in FIFO order based purely on tree depth (number of hops from root). Assumes all action step costs are identical ($c = 1$).
- **Uniform Cost Search (UCS / Dijkstra's Search):** Expands nodes in order of accumulated path cost $g(n)$ using a priority queue. Works on arbitrary, non-uniform positive edge step costs $c(s, a, s') \geq \epsilon > 0$.

2. Complexity Derivation:

- Let C^* be the optimal path cost to the goal, and let ϵ be the minimum step cost between any two connected nodes.
- In the worst case, the search may explore nodes along paths up to depth $\lfloor C^* / \epsilon \rfloor$.
- **Time Complexity:** $\mathbf{O}\left(b^{1 + \lfloor C^* / \epsilon \rfloor}\right)$ If all step costs are equal ($\epsilon = c$), then $C^* / \epsilon = d$, and time complexity simplifies to $O(b^d)$, matching standard BFS.
- **Space Complexity:** $\mathbf{O}\left(b^{1 + \lfloor C^* / \epsilon \rfloor}\right)$ UCS must store all explored and frontier nodes in the priority queue to maintain global minimum-cost ordering.

3. Optimality Condition:

UCS is guaranteed optimal **if and only if all step costs are strictly positive** ($c \geq \epsilon > 0$). If zero-cost or negative-cost cycles exist, UCS can enter infinite loops without advancing towards the goal.

Ramaiah SEE Archive · Model Question Paper [8 Marks]

"Prove the optimality of A^* search for: (a) Tree search under an admissible heuristic, (b) Graph search under a consistent (monotonic) heuristic."

Practice Problem:

Show Complete Step-by-Step Model Answer

Solution:**(a) Optimality Proof for Tree Search (A^* with Admissible Heuristic):**

1. Suppose G_2 is a suboptimal goal node on the OPEN list, so $g(G_2) > C^*$, where C^* is the optimal solution cost. Since G_2 is a goal node, $h(G_2) = 0$, giving $f(G_2) = g(G_2) > C^*$.
2. Let n be an unexpanded node on the OPEN list that lies on an optimal path to the optimal goal G .
3. Because $h(n)$ is admissible ($h(n) \leq h^*(n)$):

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$$

4. Combining both inequalities:

$$f(n) \leq C^* < f(G_2)$$

5. Since $f(n) < f(G_2)$, the priority queue will pop node n before G_2 . By induction, all nodes on the optimal path are expanded before G_2 can ever be popped. Therefore, A^* tree search never selects a suboptimal goal. **lacksquare**

(b) Optimality Proof for Graph Search (A^* with Consistent Heuristic):

1. A heuristic is **consistent (monotonic)** if for every node n and every successor n' generated by action a :

$$h(n) \leq c(n, a, n') + h(n')$$

2. Adding $g(n)$ to both sides:

$$g(n) + h(n) \leq g(n) + c(n, a, n') + h(n')$$

$$f(n) \leq g(n') + h(n') = f(n')$$

3. Thus, $f(n)$ is **monotonically non-decreasing along any path**.
4. Consequently, whenever A^* selects a node n for expansion from the OPEN list, the path found to n is guaranteed to be optimal. No previously expanded node on the CLOSED list ever needs to be reopened, guaranteeing graph search optimality. **lacksquare**

Ramaiah AI Question Bank · Stochastic Optimization [6 Marks]

"Explain the operation of Simulated Annealing. Detail how the cooling schedule and Boltzmann probability $P = e^{\Delta E/T}$ prevent the search from getting trapped in local maxima."

Practice Problem:

Show Complete Step-by-Step Model Answer

Solution:**1. Inspiration & Algorithmic Framework:**

Adapted from metallurgical annealing (gradual cooling of molten metals to reach low-energy crystalline states), Simulated Annealing escapes local maxima by combining greedy hill-climbing with stochastic exploration.

2. Downhill Move Acceptance Rule:

- At state s , pick a random neighboring state s' . Compute energy difference $\Delta E = E(s') - E(s)$.
- If $\Delta E > 0$ (uphill move towards improvement), the move is **always accepted**.
- If $\Delta E \leq 0$ (downhill move), the move is accepted probabilistically with **Boltzmann probability**:

$$P = e^{\Delta E/T}$$

where $T > 0$ is the current temperature parameter.

3. Role of Temperature (T) and Cooling Schedule:

- **Initial High Temperature ($T \gg 0$):** When T is very large, $\Delta E / T \approx 0$ implies $P \approx e^0 = 1$. The search accepts almost all downhill moves, behaving like a random walk that freely explores the entire landscape without getting trapped.
- **Cooling Progression ($T_{k+1} = \alpha T_k, 0.8 \leq \alpha \leq 0.99$):** As T decreases, the probability P of accepting worse moves drops exponentially. The agent becomes increasingly selective, accepting only shallow downhill moves.
- **Freezing Phase ($T \rightarrow 0$):** As $T \rightarrow 0$, $P \rightarrow 0$. Downhill moves are strictly rejected, and the algorithm smoothly degenerates into standard greedy hill-climbing, locking cleanly into the nearest global peak.

Verified High-Yield Practice Problems

PRACTICE

Test your problem-solving skills with these exam-caliber problems. All numerical calculations and algorithmic steps are verified against `audit/verify/ai/verify_ai_u2.py`:

PRACTICE 1 · BFS EXPONENTIAL MEMORY ANALYSIS

In a search tree with branching factor $b = 10$ and depth $d = 6$, calculate the number of nodes stored in memory by BFS if each node requires 100 bytes. What happens at depth $d = 8$?

Practice Problem:

Show Step-by-Step Solution

Solution:

1. Memory Consumption at Depth $d = 6$:

$\text{Total Nodes Stored} \approx b^d = 10^6 = 1,000,000 \text{ nodes}$
 $\text{Memory} = 1,000,000 \times 100 \text{ bytes} = 100,000,000 \text{ bytes} \approx \mathbf{100 \text{ MB}}$

2. Memory Consumption at Depth $d = 8$:

$\text{Total Nodes Stored} \approx 10^8 = 100,000,000 \text{ nodes}$
 $\text{Memory} = 10^8 \times 100 \text{ bytes} = 10,000,000,000 \text{ bytes} \approx \mathbf{10 \text{ GB}}$

Takeaway: BFS memory scales exponentially ($O(b^d)$). In contrast, IDDFS achieves the exact same completeness and optimality at depth 8 while requiring only linear memory $O(bd) = 10 \times 8 \times 100 \text{ bytes} = \mathbf{8 \text{ KB}}$!

PRACTICE 2 · A* GRAPH SEARCH NUMERICAL TRACE

Trace A* search on the graph with nodes S, A, B, C, D, G , edge step costs

$c(S, A) = 1, c(S, B) = 4, c(A, B) = 2, c(A, C) =$

$5, c(B, C) = 2, c(C, D) = 2, c(D, G) = 1, c(C, G) = 4$, and

heuristic values $h(S) = 7, h(A) = 6, h(B) = 4, h(C) = 2, h(D) = 1, h(G) = 0$. Find the optimal path and cost.

Practice Problem:

Show Step-by-Step Solution

Solution:

Step-by-step priority queue expansion log (verified via
audit/verify/ai/verify_ai_u2.py):

1. **Step 1:** Pop S ($g = 0, h = 7, f = 7$). Successors: A ($g = 1, h = 6, f = 7$), B ($g = 4, h = 4, f = 8$).
2. **Step 2:** Pop A ($g = 1, h = 6, f = 7$). Successors: B via A ($g = 3, h = 4, f = 7$), C via A ($g = 6, h = 2, f = 8$).
3. **Step 3:** Pop B ($g = 3, h = 4, f = 7$). Successor: C via B ($g = 5, h = 2, f = 7$).
4. **Step 4:** Pop C ($g = 5, h = 2, f = 7$). Successors: D ($g = 7, h = 1, f = 8$), G ($g = 9, h = 0, f = 9$).
5. **Step 5:** Pop D ($g = 7, h = 1, f = 8$). Successor: G ($g = 8, h = 0, f = 8$).
6. **Step 6:** Pop G ($g = 8, h = 0, f = 8$). Goal reached!

Optimal Path: SoAoBoCoDoG with **Optimal Cost = 8.0**.

PRACTICE 3 · HEURISTIC CONSISTENCY PROOF

Prove mathematically by mathematical induction that every consistent (monotonic) heuristic is necessarily admissible.

Practice Problem:

Show Step-by-Step Solution

Solution:**Proof:**

1. By definition of consistency, for any state n and successor n' via action a :

$$h(n) \leq c(n, a, n') + h(n')$$

2. Let $n = n_0 n_1 n_2 \dots n_k = G$ be the optimal path from node n to goal G .

3. Applying the consistency inequality along this optimal chain:

$$h(n_0) \leq c(n_0, a_1, n_1) + h(n_1)$$

$$h(n_0) \leq c(n_0, a_1, n_1) + c(n_1, a_2, n_2) + h(n_2)$$

$$h(n_0) \leq \sum_{i=0}^{k-1} c(n_i, a_{i+1}, n_{i+1}) + h(G)$$

4. Since G is a goal state, $h(G) = 0$.

5. The sum of optimal step costs is by definition the true optimal cost $h^*(n_0)$:

$$h(n_0) \leq h^*(n_0) + 0 = h^*(n_0)$$

6. Therefore, $h(n) \leq h^*(n)$ for all nodes n , proving h is admissible. **lacksquare**

PRACTICE 4 · 8-PUZZLE HEURISTIC DOMINANCE

Calculate the Manhattan distance heuristic h_2 for the 8-puzzle start state where tile 5 is at $(1, 3)$ and goal is $(3, 2)$. Prove why h_2 strictly dominates h_1 (misplaced tiles).

Practice Problem:

Show Step-by-Step Solution

Solution:**1. Manhattan Calculation for Tile 5:**

$$d_M(5) = |x_1 - x_2| + |y_1 - y_2| = |1 - 3| + |3 - 2| = 2 + 1 = 3$$

2. Dominance Proof ($h_2(n) \geq h_1(n)$):

- For any misplaced tile t , it must move at least 1 square to reach its goal position
 $\implies d_M(t) \geq 1$.
- For any tile already in its goal position, $d_M(t) = 0$.
- Therefore:

$$h_2(n) = \sum_{t=1}^8 d_M(t) \geq \sum_{t \text{ misplaced}} 1 = h_1(n)$$

- Since $h_2(n) \geq h_1(n)$ for all states n , h_2 **dominates** h_1 . As a consequence, A^* using h_2 will expand a subset of the nodes expanded by A^* using h_1 , saving substantial computation.

PRACTICE 5 · BOLTZMANN PROBABILITY CALCULATION

In Simulated Annealing with $\Delta E = -5.0$, compute the exact acceptance probability $P = e^{\Delta E/T}$ at $T = 1000\text{extK}$, 100extK , 10extK , and 1extK .

Practice Problem:

Show Step-by-Step Solution

Solution:

Computed via `audit/verify/ai/verify_ai_u2.py`:

- At $T = 1000\text{extK}$: $P = e^{\{-5 / 1000\}} = e^{\{-0.005\}}$ pprox $\mathbf{0.995012}$ ext{ (99.50%)}}.
- At $T = 100\text{extK}$: $P = e^{\{-5 / 100\}} = e^{\{-0.05\}}$ pprox $\mathbf{0.951229}$ ext{ (95.12%)}}.
- At $T = 10\text{extK}$: $P = e^{\{-5 / 10\}} = e^{\{-0.5\}}$ pprox $\mathbf{0.606531}$ ext{ (60.65%)}}.
- At $T = 1\text{extK}$: $P = e^{\{-5 / 1\}} = e^{\{-5.0\}}$ pprox $\mathbf{0.006738}$ ext{ (0.67%)}}.

Conclusion: At high T , worse moves are accepted with 99.5% probability (allowing escapes from local maxima). Near $T = 1$, the acceptance rate drops below 1%, forcing convergence.

PRACTICE 6 · UCS GOAL TEST TIMING & ZERO COSTS

Why must Uniform Cost Search apply the goal test when a node is selected for **EXPANSION** (popped from priority queue), rather than when it is **GENERATED** (inserted into queue)?

Practice Problem:

Show Step-by-Step Solution

Solution:**Answer:**

- If the goal test were applied when a node is generated, UCS could stop immediately upon discovering the first path to the goal.
- However, that first path might have a high step cost (e.g., *SoG* with cost 10). A much cheaper path might exist (e.g., *SoAoBoG* with total cost $1 + 1 + 1 = 3$), but its intermediate nodes have not yet been expanded!
- By testing for the goal only when a node is **popped for expansion**, UCS guarantees that all nodes with path cost less than $g(G)$ have already been expanded. This ensures that when G is popped, no cheaper path to G can possibly exist, preserving mathematical optimality.